

李沐21年的逐段精读 transformer

2026年8月30日 星期日 下午4:31

要做的仍是从 sequence 到 sequence 的生成,

中间加了个 Attention 机制, 但是 Attention 机制本身也不是他们的原创

只不过他们做的太好了 属于发扬光大

刻意避免 用到 循环和卷积 实际上, 为了克服循环和卷积的性能问题, 创造的transformer成为了与其并列的网络类型

result 没有特别夸张 也就 $26.4 \rightarrow 28.4$ BLEU 一开始针对的任务是 机器翻译 应用到其他领域是后来的事情

代码还是放 Abstract 最后 比较好, 虽然这篇不是这么做的

background

讲的是 RNN $h_{t-1}, input_t \rightarrow h_t, output_t$ 并行, 内存问题

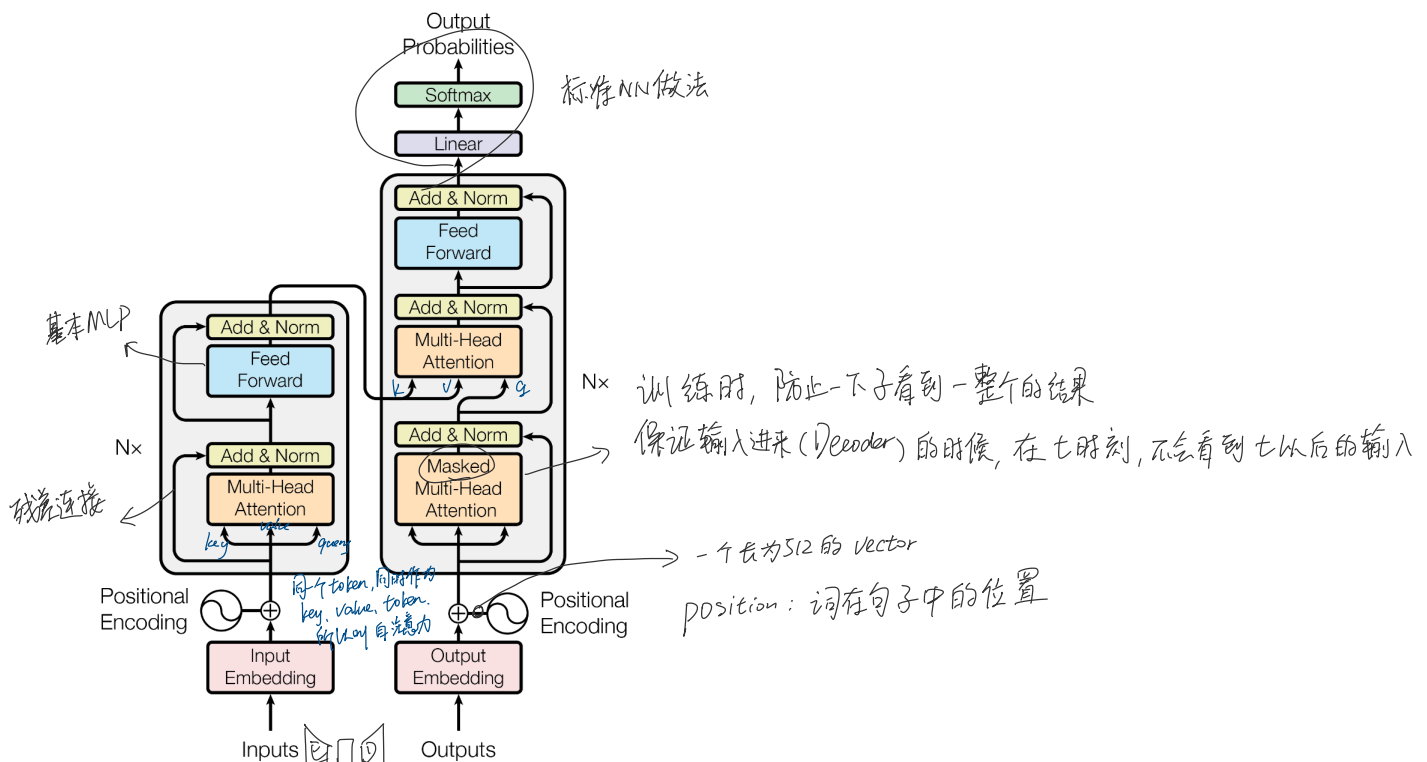


卷积有上下文联想困难问题

Encoder-Decoder结构大意

$(x_1, \dots, x_n) \rightarrow z = (z_1, \dots, z_n) \rightarrow (y_1, \dots, y)$
symbol continuous

Auto-Regressive 自回归 输出又拿回来成为输入, 导致要一步步生成输出

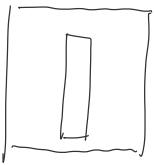


(shifted right)
一个个右移

"Simple, position-wise fully connected feed-forward network" $\Rightarrow$ MLP

$$\text{LayerNorm}(x + \text{Sublayer}(x))$$

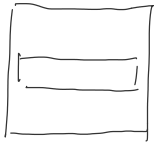
(batch norm)



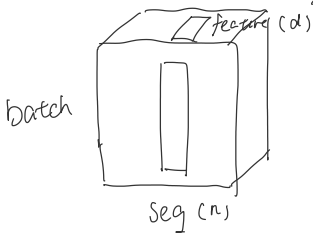
Norm 就是均值 $\rightarrow 0$, 方差 $\rightarrow 1$

batch 对每一个特征做 norm, 不论 2 维或 3 维, 都是从 feature 上切片, 去做 norm

(layer norm)



$\rightarrow$ 此文设为 512



关键点: 自注意力 self-attention

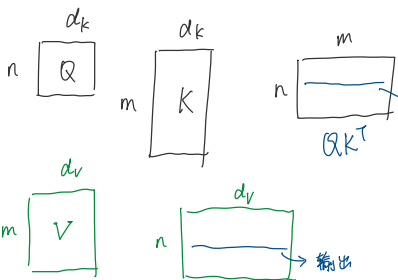
"An attention function can be described as mapping a query and a set of key-value pairs to an output"

$\frac{1}{\sqrt{d_k}}$
scaled dot-product attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$\rightarrow$ 对每一行做 softmax

$\rightarrow$ 防止值过大 (不清楚为什么) $\Rightarrow$ 防止方差太大, 促进平滑



$\rightarrow$ 一个 query, 对所有 key 的内积值

$\rightarrow$ 输出

Multithead attention. 可以理解为并行处理, 每个头处理不同的信息

"投影": 用一个线性层, 设置所需的输出层大小. 层间维度不同加个linear, 跟胶水一样, 就认为是有效连接了

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

投影到一个 W^O

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

对每一个头, 通过一个不同的, 可学习的 W_Q, W_K, W_V 投影到一个低维上, 再做 Attention

3.2 可理解为 K-V 已经包含了所有编码后的信息. q 只需要不断询问 - 得出结论 - 根据结论再次询问

可以去听李宏毅版本 这部分讲的更清楚一点

point-wise

"applied to each position separately and identically"

position: 输入序列中, 每个词对应的序列中的位置

它把一个 MLP, 对每一个词作用一次 $\Rightarrow$ point-wise

说白了就是一个 MLP, 作用在最后一个"维度"?

$$\text{FFN}(x) = \max(0, \underbrace{xW_1 + b_1}_{\text{linear}}) \underbrace{W_2 + b_2}_{\text{ReLU 激活层}}_{\text{linear}}$$

512 $\rightarrow$ 2048 2048 $\rightarrow$ 512

Compare with RNN

图解略

★ 都是在于如何有效使用序列信息

原始 attention 无法处理时序信息.

$\Rightarrow$ 加入 PE

一个 token 在 embedding 层表示为一个长为 512 的向量

同样, 用一个长为 512 的 vector 编码的数字去表示位置

$$\text{PE}(\text{pos}, z_i) = \sin(\text{pos} / 10000^{z_i / d_{\text{model}}})$$

or

$$\cos(\text{pos} / 10000^{z_i / d_{\text{model}}})$$

$$r_t(pos, 2i+1) = w \cdot (pos / 10000 \cdot \dots)$$

时序向量与 embedding 向量相加, 就达到了将时序信息加入数据的做法

(512)

数学上, CNN 是可以写为一种受限 Self-Attention 的形式

10万步, 是指 epochs 吗? batch?

调参. $\gamma, \beta_{1,2}, \epsilon$ 这些东西也没咋调. result(?) 里提一嘴就行

Regularization

└ Residual Dropout

└ Label Smoothing

Table 2

对比} 超参数对结果的影响